



Murphy's Bench — What It Does

Murphy's Bench is self-hosted repair-shop software for small field-service businesses and MSPs. It runs on your own server, on your own network. You own the data, there's no per-seat SaaS bill, and nothing about your clients leaves your control.

This document is for a technician or shop owner sizing it up: **what can it actually do for you day to day**. It doesn't cover how it's built or how to install it — see `INSTALL.md` for that.

The core idea: Ticket → Work Order

Most repair work follows one shape, and Murphy's Bench is built around it:

A request comes in (a **Ticket**) → you talk to the customer → it becomes actual repair work (a **Work Order**) → you do the work and track it → it's done and ready to bill.

The two halves are deliberately separate:

- A **Ticket** is the conversation — the customer-facing thread.
- A **Work Order** is the job — the bench/onsite work, parts, time, and checklist.

A ticket can become a work order with one click, and the two stay linked so you can always jump between "what the customer was told" and "what was actually done." Crucially, **closing a work order never auto-closes the ticket** — a human decides the customer interaction is finished. No job quietly falls off the radar.

Tickets — the customer conversation

- **Email in, ticket out.** Emails sent to your support address automatically become tickets. Customer replies thread back into the same ticket.
 - **Threaded replies**, like an email client — staff-to-customer messages, internal notes only your team sees, and inbound customer replies are visually distinct so you always know who said what.
 - **One voice to the customer.** The ticket is the single channel the customer sees. A bench tech who needs the customer contacted messages the ticket owner internally rather than emailing the customer directly — so the customer never gets two people talking at them.
 - **HTML email with your branding** — your logo and colors on outbound mail, with signatures.
 - **Auto-responder** so a customer knows their email was received.
 - **Quoted history folds away** — long email chains collapse so the new message is what you actually read.
 - **Status tracking** (New → Open → In Progress → Waiting on Customer → Resolved → Closed) with colors, and you can define your own custom statuses.
 - **Collision avoidance** — if a teammate already has a ticket open, you're warned before you both step on it.
 - **Link related tickets** together (duplicates, related issues).
-

Work Orders — the actual job

- **In-shop, onsite, or remote** — each work order knows which, and onsite jobs get mileage tracking.
 - **Built-in stopwatch** to log time as you work, plus quick-entry labor lines.
 - **Checklists** — pre- and post-repair task lists, scoped by device type, so nothing gets skipped.
 - **Work Performed log** — a running, timestamped record of what you did, which becomes the customer's repair report.
 - **Printable Repair Report** — a clean, professional printout with your company info, the device, the work done, and signature lines.
 - **Device credentials, encrypted** — store the login/password/PIN for the machine you're working on, masked by default, with every reveal logged.
-

Clients, contacts, and devices

Everything is organized around the **client**, the way a shop actually thinks:

- A client has multiple **contacts** (people), each with multiple phone numbers.
 - A client has multiple **devices**, each with its own repair history.
 - Residential vs. business clients are handled differently where it matters.
 - From a client's page you see their whole history and can start a new work order for a specific person and device in a couple of clicks.
-

Knowing where things stand

- **Dashboard** with color-coded tiles — what's active, what's waiting, what's overdue, what's done — at a glance.
 - **Per-technician views** — techs see their own work plus the unclaimed pool; admins see everything. Techs **claim, transfer, and escalate**; dispatchers **assign**.
 - **Escalation levels** — push a ticket up to a more senior tech without losing the current owner, so nothing disappears into a black hole.
 - **In-app notifications** — a sidebar bell tells a tech when a teammate needs something from them.
 - **Saved queues** — build your own filtered views of tickets and reuse them.
-

Deadlines and accountability (SLAs)

- **SLA plans** set response deadlines per type of work.
 - Overdue tickets are flagged, and acknowledging one **requires a note** — so there's an audit trail of why something slipped, not just that it did.
-

Billing (lightweight, on purpose)

Murphy's Bench tracks billing **state** — it is not a full accounting package, and it doesn't try to be.

- Each work order has a billing status: uninvoiced, invoiced, paid, paid-direct (cash/walk-in), or disputed.
 - **Outstanding balances** roll up per client.
 - **CSV export** of invoices and billing so it drops into whatever accounting system you already use. (A direct Invoice Ninja bridge is planned.)
-

Reporting

- A reports page with several built-in reports — workload, technician performance (completion rate, average resolution time), billing summaries, mileage, and more — with charts.
 - **Every report exports** to CSV, print, or PDF.
-

Knowledge base

- A built-in **KB** for troubleshooting guides, how-tos, vendor notes, and internal-only articles.
 - Articles are written in Markdown and render with proper formatting.
 - Some articles can be marked internal/restricted.
-

Security and access control

- **Multi-factor authentication (TOTP)** with authenticator apps, backup codes for admins, and admin-driven MFA reset for a lost device.
- **Roles and permissions** — fine-grained control over who can see and do what (including who may reveal stored credentials).
- **Encrypted credential storage** for both device and organization-level secrets (Wi-Fi, portal

logins, etc.), with a full **access log** — every view of a secret is recorded.

- **Login lockout** protection against password guessing.
 - The whole app is login-only and built to sit behind HTTPS.
-

Why self-hosted

- **Your data stays yours** — on your hardware, on your network. Sensitive customer and credential data never sits on someone else's cloud.
 - **No per-seat subscription.** Add techs without adding to a monthly bill.
 - **You control updates and uptime.** Nothing changes under you on a vendor's schedule.
 - Accessible securely from anywhere via a **Cloudflare Tunnel** when you want remote access, without exposing your network.
-

Honest about where it is

Murphy's Bench is in **active daily production use** at the shop that builds it, and it's being hardened for use by others. It's deliberately scoped: it does ticketing, work orders, devices, mileage, email, credentials, and reporting **well**, rather than trying to be everything. It is not (yet) a customer self-service portal, a full accounting system, or a multi-tenant SaaS — those are out of scope by choice.

If you run a small shop and want repair-tracking software you fully control, this is built for exactly that.